# BULLION LEDGER PWA - TECHNICAL ARCHITECTURE SUMMARY

*Two-Page Executive Architectural Specification | Frontend, Backend, Multi-Tier Storage & Algorithmic Engines*

## 1. END-TO-END APPLICATION ARCHITECTURE & SYSTEM OVERVIEW

Bullion Ledger is an enterprise-grade Progressive Web Application (PWA) built for high-precision physical precious metal portfolio accounting, live valuation, and multi-tenant cloud distribution. Designed with an offline-first architecture, it provides instant sub-10ms screen paints on mobile devices, resilient browser caching for 5,666+ historical benchmark records, and zero-downtime synchronization to managed Supabase cloud PostgreSQL with Netlify edge hosting.

## 2. FRONTEND CLIENT PLATFORM & PWA RUNTIME

- **React 19 + TypeScript 5.7:**
  Component-driven single page application built with strict static typing to guarantee mathematical precision across bullion purities (995, 999, 916), gross/net gram weights, and multi-lot tax calculations.
- **Vite 6 + Rollup Bundler:**
  High-efficiency bundler delivering sub-2-second production builds. Implements custom Rollup chunk-splitting to isolate historical archives and vendor libraries, keeping core JS bundle below 240 KB.
- **Tailwind CSS v3 (JIT):**
  Utility-first design system implementing a mobile-first responsive grid for smartphones, tablets, and desktops. Customized luxury bullion palette with gold primary accents (#B88B2A) and neutral slate backgrounds.
- **Service Worker (sw.js):**
  Implements Stale-While-Revalidate caching for all application assets. Enables full offline functionality, fast asset delivery, and automatic cache invalidation during updates without stale asset locks.
- **Web App Manifest:**
  Configured via manifest.webmanifest to support standalone full-screen display mode, native app orientation, and seamless 'Add to Home Screen' installation on iOS Safari and Android Chrome.
- **Hash-Based Deep Linking:**
  Synchronizes active navigation state with URL hash routes (#dashboard, #purchases, #holdings, #prices, #settings), eliminating screen bounce and ensuring bookmarkable view persistence across refreshes.

## 3. CLOUD BACKEND, SECURITY & HOSTING INFRASTRUCTURE

- **Supabase Cloud PostgreSQL:**
  Enterprise-grade managed relational database exposing high-performance REST APIs over PostgREST with compound indices on transaction dates, metal types, purities, and soft-delete flags.
- **PostgreSQL Row Level Security (RLS):**
  Kernel-level database authorization: Public Read allows external viewers to inspect holdings, valuations, and charts without logging in; Super User Write strictly limits INSERT/UPDATE/DELETE to kosamgo@gmail.com.
- **Supabase GoTrue Auth Service:**
  Multi-modal authentication engine supporting Google OAuth 2.0 (Google Cloud Console integration), 1-Click Passwordless Email Magic Link & 6-digit OTP confirmation, and traditional Email/Password credentials.
- **Netlify Global Edge Delivery:**
  Automated CI/CD from GitHub main branch with Single Page Application rewrites (_redirects) and zero-downtime global content distribution network (CDN) ensuring sub-second worldwide latency.

## 4. MULTI-TIER DATA STORAGE & RESILIENCE MODEL

| STORAGE TIER | TARGET SYSTEM | PERSISTENCE & OPERATIONAL ROLE |
|---|---|---|
| **Tier 1: Cloud Master** | Supabase PostgreSQL | Master database for purchases, rates, settings. Indexed for cloud synchronization. |
| **Tier 2: Client IndexedDB** | HTML5 IndexedDB (IDB) | Persistent browser database storing 5,666+ historical records for offline charts. |
| **Tier 3: Fast Cache** | Browser LocalStorage | Synchronous key-value cache enabling sub-10ms UI bootstrap before async DB initializes. |
| **Tier 4: Static Archive** | defaultRates.json | Pre-bundled dataset of 5,666 verified daily rates (2019-2026) merged with cloud updates. |

# SECTION 2: KEY PWA FUNCTIONS & PROGRAMMING PLATFORMS

*Algorithmic Implementation, Valuation Engine, Newton-Raphson Solver & Data Ingestion*

## 1. DYNAMIC VALUATION & RESALE ENGINE (src/utils/valuationEngine.ts)

**Calculates true portfolio worth via dual-accounting methodology:**

- Reference Benchmark Value = Gross Pure Metal Weight x Official Exchange Settlement Spot Rate.
- Estimated Resale Value = Reference Value x (1 - Resale Markdown Haircut Pct) [Baseline: 3.0% Gold, 3.0% Silver].
- O(1) Composite Rate Indexing: Generates composite memory lookup keys (YYYY-MM-DD_METAL_PURITY) to evaluate thousands of lots in sub-millisecond execution cycles.
- Trading Day Carry-Forward: Bridges market closures (weekends, holidays) by dynamically fetching the preceding trading day's closing benchmark with audit tags.

## 2. INSTITUTIONAL NEWTON-RAPHSON XIRR SOLVER (src/utils/xirr.ts)

**Computes exact annualised Internal Rate of Return (XIRR) across irregular, multi-year cash outflows:**

- Mathematical Formula: Solves sum[ Ci / (1 + r)^((di - d0)/365) ] = 0 where Ci represents cash outflows (purchase cost + GST + charges) and current liquidation resale value.
- Numerical Iteration: Applies Newton-Raphson tangent root-finding with analytic first derivatives: r_{k+1} = r_k - f(r_k)/f'(r_k). Achieves precision epsilon < 10^-6 within 10-25 iterations.
- Robust Boundary Safeguards: Implements zero-division protection for same-day investments (di = d0) and adaptive step dampeners to prevent oscillation during extreme market swings.

## 3. AUTOMATED IBJA BENCHMARK RATES INGESTION (src/services/ibjaRateService.ts)

**Sources daily institutional closing settlement benchmarks directly from the India Bullion and Jewellers Association:**

- Headless Browser Extraction: Directly parses official IBJA tables in client memory with zero commercial API fees, external proxies, or private token requirements.
- Standardized Normalization: Standardizes heterogeneous trade units into uniform Rs/gram values: Gold 999 (Rs/10g / 10), Gold 995 (Rs/10g / 10), and Silver 999 (Rs/kg / 1000).
- Provenance & Session Auditing: Automatically attaches trading session tags (PM), GST exclusion flags, and ISO timestamps to guarantee data integrity across devices.

## 4. SPREADSHEET INGESTION & DATA PORTABILITY (src/utils/purchaseExcel.ts)

**High-performance invoice parsing and disaster recovery backup platform powered by SheetJS (xlsx):**

- Fuzzy Column Matching: Intelligently identifies merchant invoice columns ('Date', 'Invoice #', 'Metal', 'Weight', 'Rate', 'GST', 'Charges') across diverse jeweler billing formats.
- Purity Normalization: Converts trade nomenclature ('24K', '22K', '99.5%', '91.6%') into standardized decimal purities (999, 995, 916).
- Cryptographic Batch Undo: Assigns unique batch_id strings to bulk uploads, enabling 1-click rollback of any multi-lot import without affecting prior data.
- Full JSON Schema Backup: Exports and restores complete portfolio snapshots (purchases, settings, corrections, batches).

## 5. TECHNOLOGY PLATFORM MATRIX

| ARCHITECTURE DOMAIN | IMPLEMENTATION TECHNOLOGY & ROLE |
|---|---|
| **Core Language & Engine** | TypeScript 5.7 + React 19 (Strict static typing for financial accounting) |
| **Bundler & Optimization** | Vite 6 + Rollup (Sub-2s HMR build, custom code-splitting chunks) |
| **Styling & Mobile UI** | Tailwind CSS v3 (Responsive mobile-first layout with luxury gold palette) |
| **PWA & Offline Runtime** | Service Worker (sw.js) Stale-While-Revalidate caching + Web Manifest |
| **Cloud Master Database** | Supabase PostgreSQL 15+ (Relational persistence, PostgREST REST API) |
| **Authorization & Auth** | PostgreSQL Row Level Security (RLS) + Google OAuth 2.0 / Magic Link |
| **Client-Side Storage** | HTML5 IndexedDB (5,666 rate cache) + LocalStorage (10ms bootstrap) |
| **Financial Solvers** | Newton-Raphson XIRR solver + Dual-Valuation Resale Engine |
| **Data Ingestion & Portability** | SheetJS (xlsx) Excel parser + Headless IBJA closing rates extractor |
| **Edge Hosting & CDN** | Netlify Global Edge CDN (Automated Git deployments, SPA rewrites) |